資料結構與演算法 I
(Data Structure and Algorithms I)
2025 fall

2025/10/31

Chapter 3 Stack and Queues

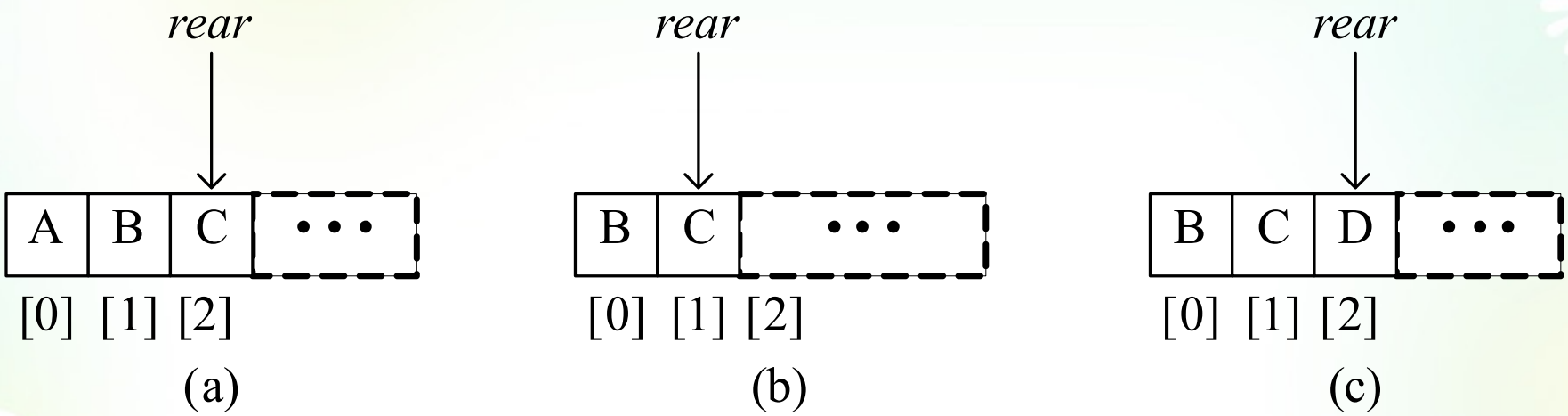


FIGURE 3.5 Queues represented with front element in queue[0]

- Since a delete or pop removes the front element, which is queue[0], each delete requires us to shift the remaining elements to the left.
- It takes $\Theta(n)$ time to pop an element from a queue that has n elements; an insertion or push, on the other hand, can be done in $\Theta(1)$ time exclusive of the time for any array resizing that may be needed

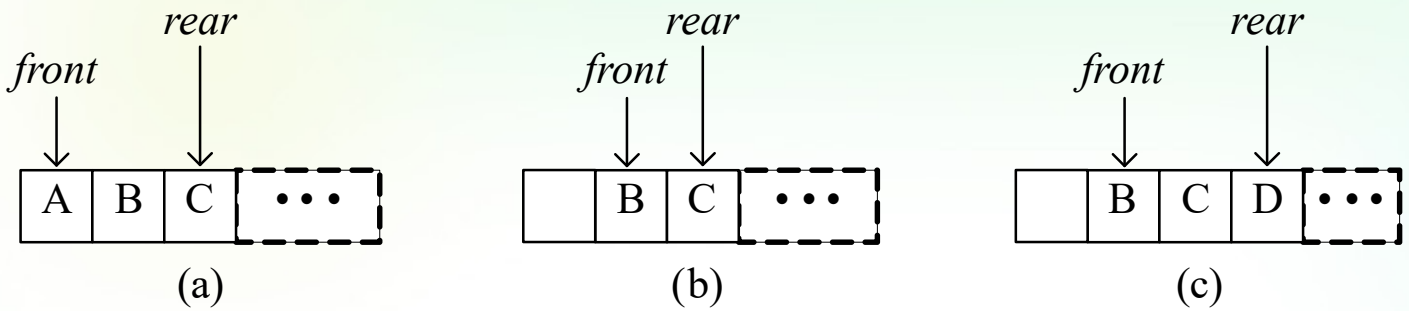


FIGURE 3.6 Queues represented with front element in queue[front]

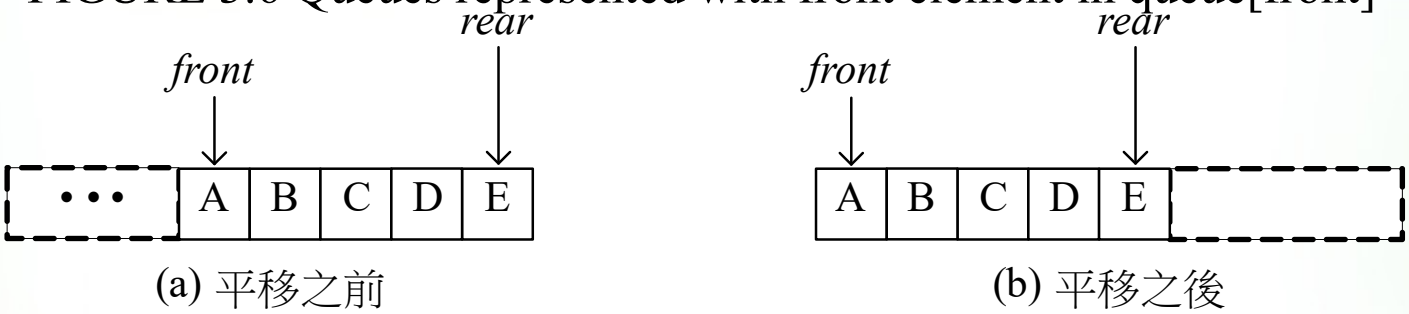


FIGURE 3.7 Shifting queue elements to the left

- To pop an element in Θ(1) time, we must relax the requirement that queue[0] contain the front element of the queue.
- A variable- front, is used to keep track the location of the front element. The queue elements are in *queue[front]*, …., *queue[rear]*.
- But it runs into a problem when ***rear*** equals ***capacity***
  - Shift all elements to the left end of the queue and create space at the right end (It takes time proportional to the size of queue)

Circular queue 環狀佇列

It is convenient to think of the array positions as arranged in a circle rather than in a straight line.

We have changed the front which points one position counterclockwise from the location of the front element in the queue.
Rear is the same!!

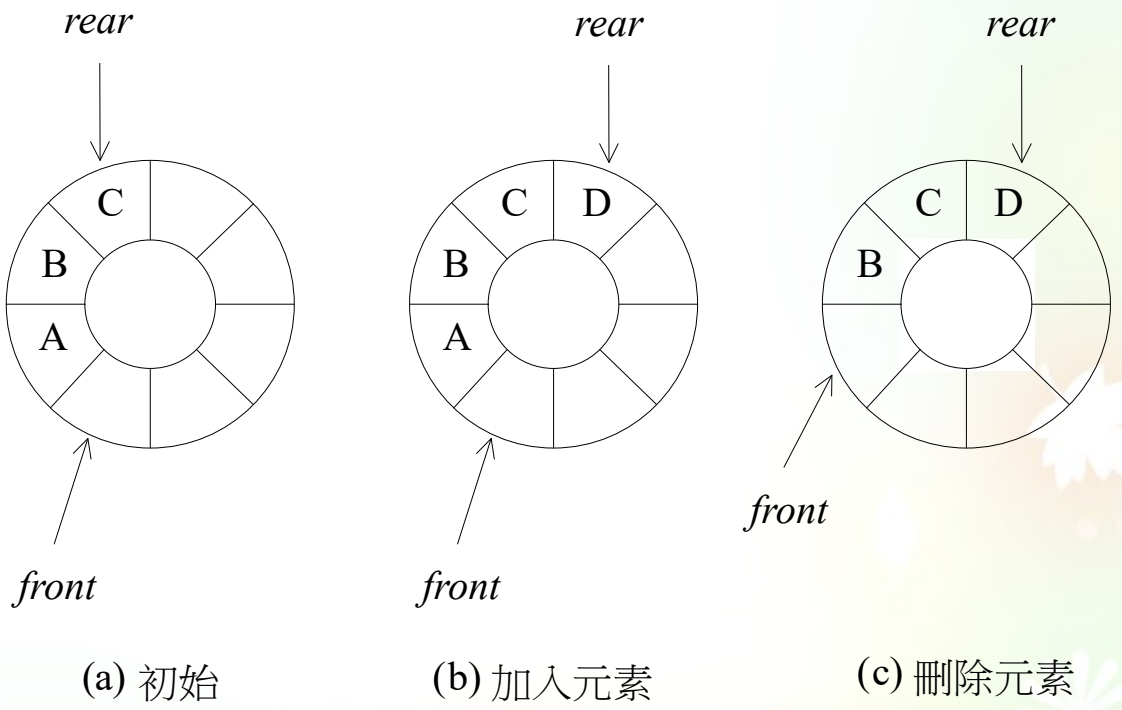
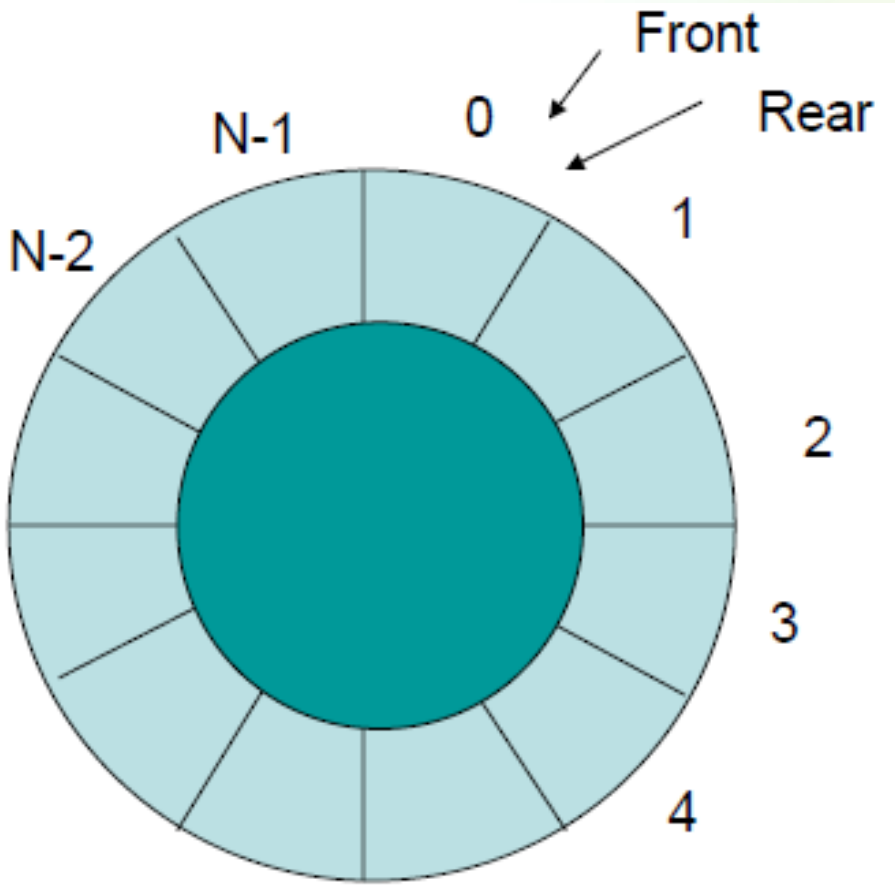


FIGURE 3.8 Circular queue

Circular queue 環狀佇列

- 一維陣列共n 個元素,製作Q(0:n-1)的環型陣列.
- n→capacity
- Q(n-1) 下一個元素是0.



Push

if (rear ==capacity-1) rear=0;
else rear++;
→ rear= (rear+1)%capacity

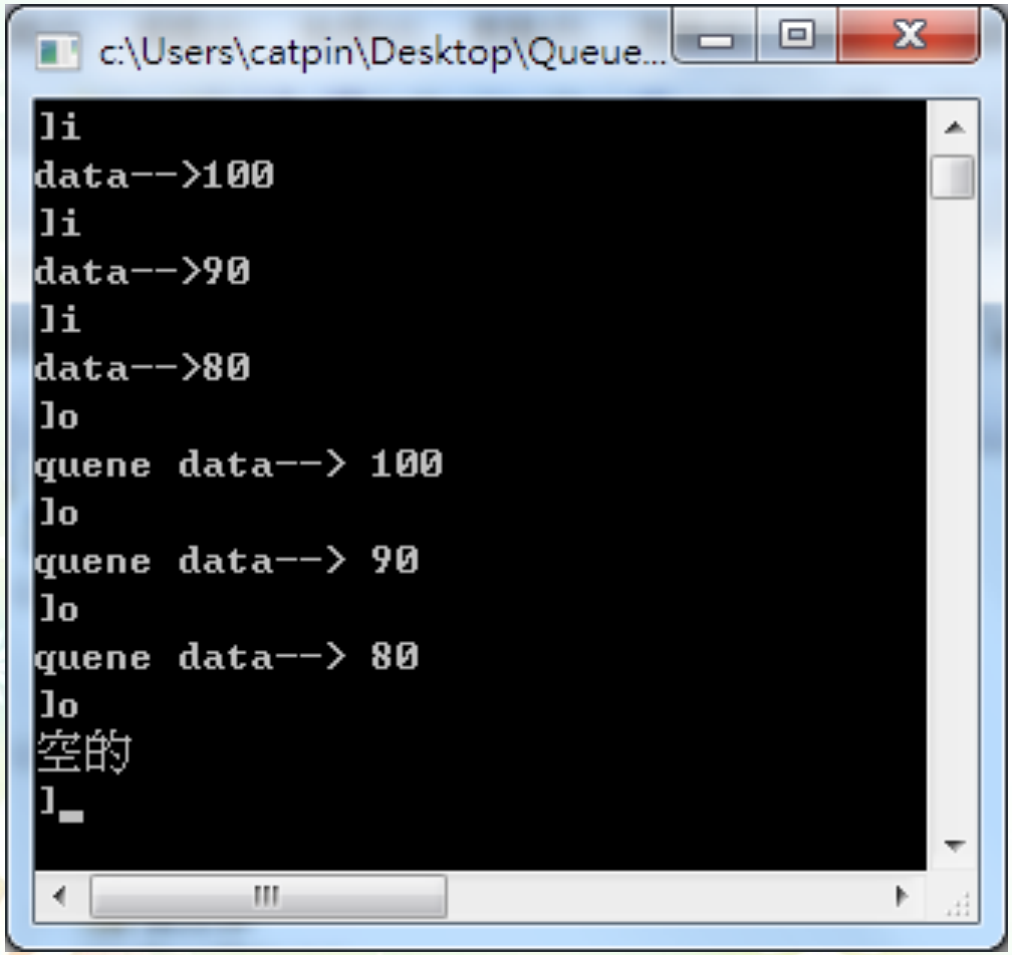
The front element of the queue is located one position clockwise from front and the rear element is at position rear

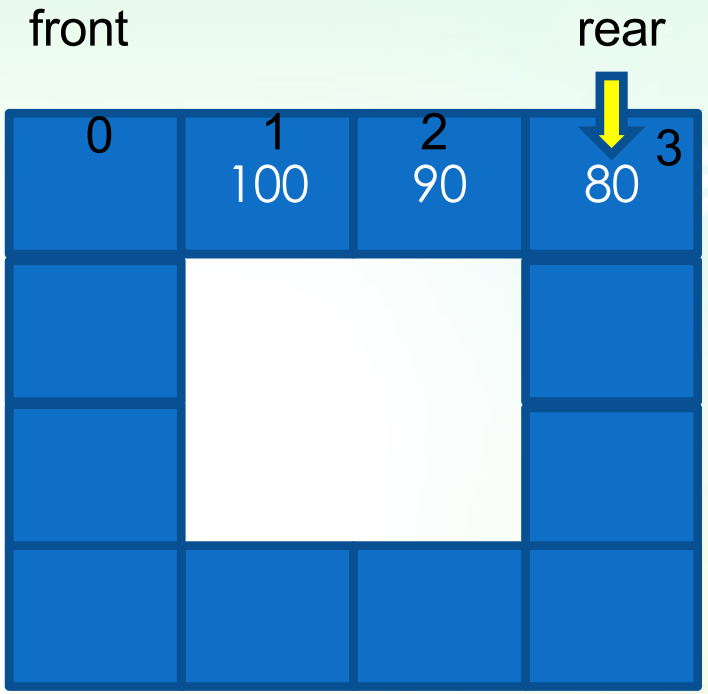
```
#include "stdafx.h"
#include "stdio.h"
#include "stdlib.h"

#define MaxSize 100
int queue[MaxSize];
int front=0, rear=0;
int enqueue(int);
int dequeue(int *);

int _tmain(int argc, _TCHAR* argv[])
{
    int c,n;
    while(printf("]"),(c=getchar())!=EOF){
      rewind(stdin);
      if(c=='i'||c=='I'){
      printf("data-->");
      scanf("%d",&n);
      rewind(stdin);
       if(enqueue(n)==-1){
            printf("已滿\n");
              }//I
       }//if
```

```
if(c=='o'||c=='O'){
if(dequeue(&n)==-1)
 printf("空的\n");
else
 printf("quene data--> %d\n",n);
                } //if
 }//while
       system("pause");
       return 0;
}
```

```
int dequeue(int *n)
{
if(rear!=front){
front=(front+1)%MaxSize;
*n=queue[front];
return 0;
}
else
return -1;
}
```

```
int enqueue(int n)
{
if((rear+1)%MaxSize!=front) {
     rear=(rear+1)%Maxsize;
     queue[rear]=n;
      return 0;
     }
   else
     return -1;}
```

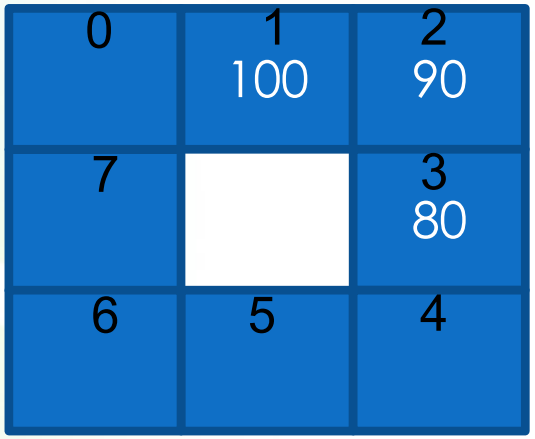




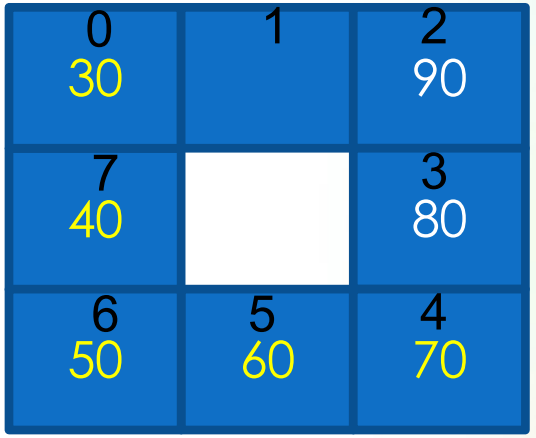
rear=front =0時 表佇列為空

事實上，程式將queue[n-1]與queue[0]連結起來，形成一個環狀陣列

rear轉一圈而位於front之前時
((rear+1)%n=front) 表示佇列已滿
但是事實上front位置沒有資料
因此佇列內之最大長度為n-1



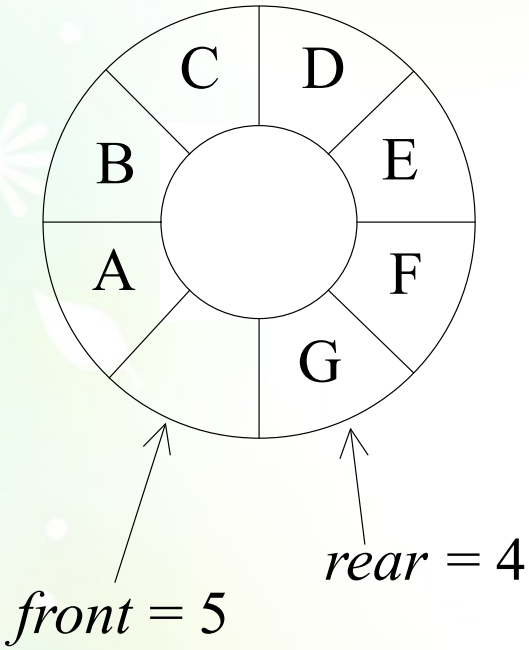
如果拿出了100
則front→1
Rear=3

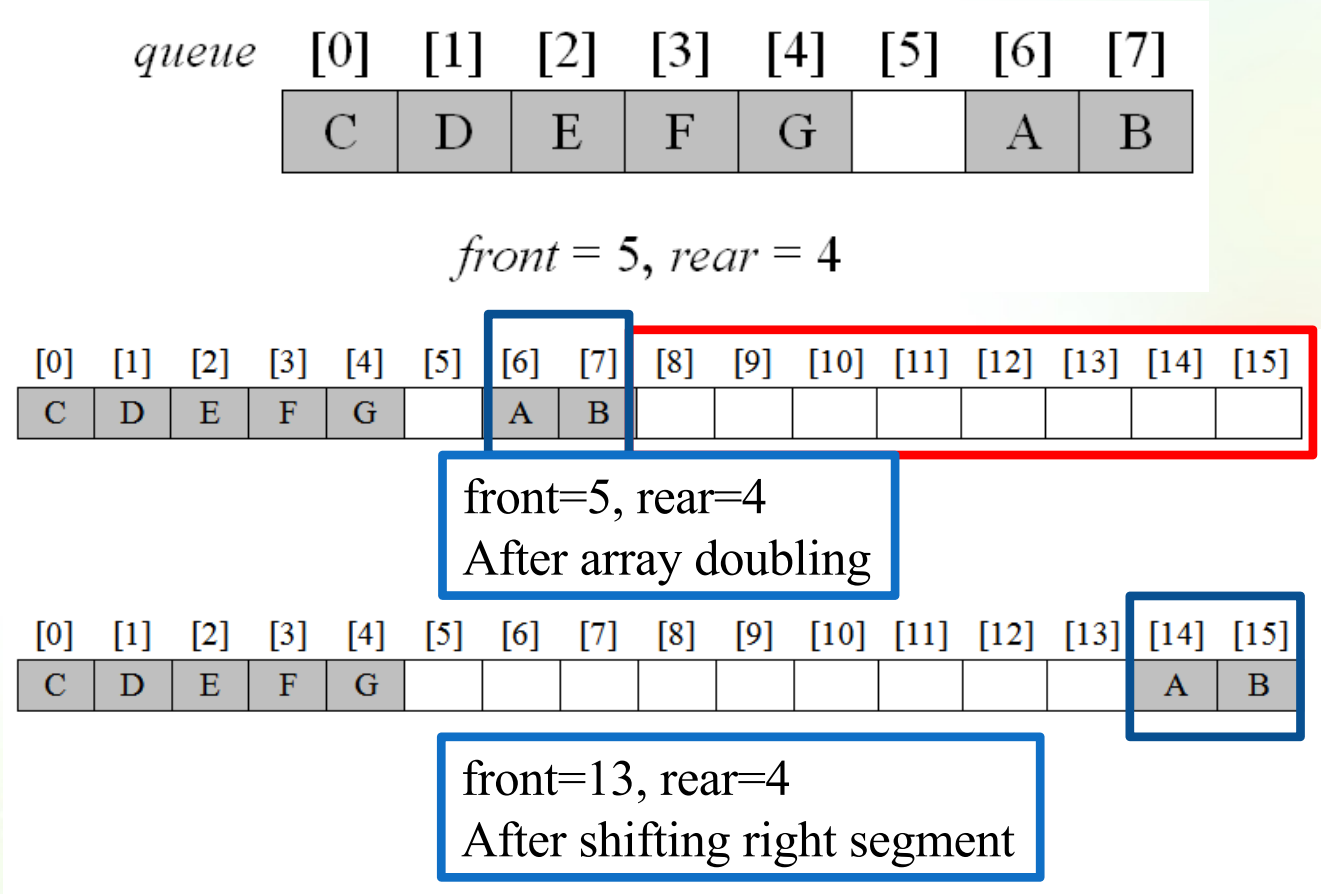


MaxSize=8;
front=0;
rear=3

如果插入70-30
Rear++; 4,5,6,7,8
8%8=0
如果還要插入20時
這時候rear=0
(0+1)%8=1=front
表示已滿，無法再新增

Double the capacity of queue- Method 1





Double the capacity of queue- Method 2

```
// 配置一個雙倍容量的陣列
T* newQueue = new T [2*capacity];
// 從queue複製至 newQueue
int start = (front + 1) % capacity;
if (start < 2)
        // no warp around
        copy (queue + start, queue + start + capacity − 1, newQueue);
else
{ // queue warps around
        copy (queue + start, queue + capacity, newQueue);
        copy (queue, queue + rear + 1, newQueue + capcity − start);
}
// 切換至 newQueue
front = 2 * capacity – 1;   因為元素由0開始擺放，所以front只好到陣列的尾巴去
rear = capacity – 2;     因為只有當內容元素為capacity-2個時，才需要加倍
capacity *= 2;
delete [] queue;
queue = newQueue;
```

queue	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]
	C	D	E	F	G		A	B

front = 5, *rear* = 4 Capacity=8 **int** *start* = (*front* + 1) % *capacity*;→**6**

Queue warps around!!!

copy (*queue* + *start*, *queue* + *capacity*, *newQueue*);
queue[6], queue[7]→newQueue[0], newQueue[1]

copy (*queue*, *queue* + *rear* + 1, *newQueue* + *capcity* − *start*);
queue[0]….queue[4]→newQueue[0+8-6], newQueue[3]…newQueue[6]

front = 2 * *capacity* – 1; →**15**
rear = *capacity* – 2; →**6**
capacity *= 2;→**16**

Inheritance 繼承

- It is used to express subtype relationships between ADTs.
- This is also referred to as the **IS-A** relationship.
  - Chair is a furniture
  - Lion is a mammal
  - Rectangle is a polygon
- How about Stack and Bag?
  - Stack and bag are the data structures into which elements can be inserted and from which the elements could be removed
  - But stack has some rules(LIFO)→ **Stack IS A Bag**

- The is-a relationship is a fundamental, conceptual relationship between the specifications of two ADTs.
  - IS-A 關係是一種基本的與概念的關係
- This relationship is not affected by the implementation of either ADTs
  - 不會影響兩種抽象資料型態的實作
- Stack IS-A Bag is true, even if their implementations are changed.
  - 堆疊是一種袋子，即使實作改變也還是可以放東西

Public Inheritance

- C++ provides a mechanism to express the IS-A relationship called ***public inheritance.***
- Bag→ base class (基本類別)
- Stack→ derived class(衍生類別)
- **virtual**, we will discuss it in next chapter. 虛擬函式
- The derived class(stack) inherits all the members(data and functions) of the base class(bag)
- Only **non-private** members of the base class are accessible to the derived class. 父類別中只有非私有函式才能夠讓延伸(子)類別存取使用

```
Class Bag
{
public:
   Bag (int bagCapacity = 10);
   virtual ~Bag( );
   virtual int Size( ) const;
   virtual bool IsEmpty( ) const;
   virtual int Element( ) const;
   virtual void Push(const int);
   virtual void Pop( );
protected:
   int *array;
   int capacity;
   int top;
};
class Stack : public Bag
{
public:
   Stack (int stackCapacity = 10);
   ~Stack( );
   int Top( ) const;
   void Pop( );
};
```

http://nknucc.nknu.edu.tw/~jwu/c/cpgch16.htm#second

- The member functions inherited by Stack from Bag have the same prototypes. This is a reuse of the interface of a base class
  - (堆疊裡面的函式都繼承於Bag，就是所謂的重複使用)
- The following code fragment illustrates how inheritance works:
  - Bag b(3);
  - Stack s(3);
  - b.Push(1);b.Push(2);b.Push(3); //use Bag::Push
  - s.Push(1); s.Push(2);s.Push(3); //use Bag::Push
  - b.Pop();//use Bag::Pop
  - s.Pop();//use Stack::Pop
  - s.Size(); //use Bag::Size
  - s.Element(); //use Bag::Element

- Queue ADT is also a subtype of Bag
- Queue can also be represented as a derived class
- Like stack and bag, queue uses an array, but unlike the stack and bag, queue requires the data member front and rear.
- The implementation of the operations isEmpty, push and pop are different from those of bag.

資料結構與演算法 I
(Data Structure and Algorithms I)
2025 fall

2024/10/31

Linked Lists

Sequential vs Linked

- Sequential representation→ items of a list are located a fixed distance apart
  - The order of elements is the same as the ordered list
- Linked representation→ items may be placed anywhere in memory
  - The two sequences need not be the same
- To access the list elements in the correct order, with each element we store the address or location of the next element in that list. Thus, associated with each data item in a linked representation is a pointer or link to the next item.
- A linked list is comprised of **nodes**; each node has **zero or more data fields** and **one or more link or pointer field**.

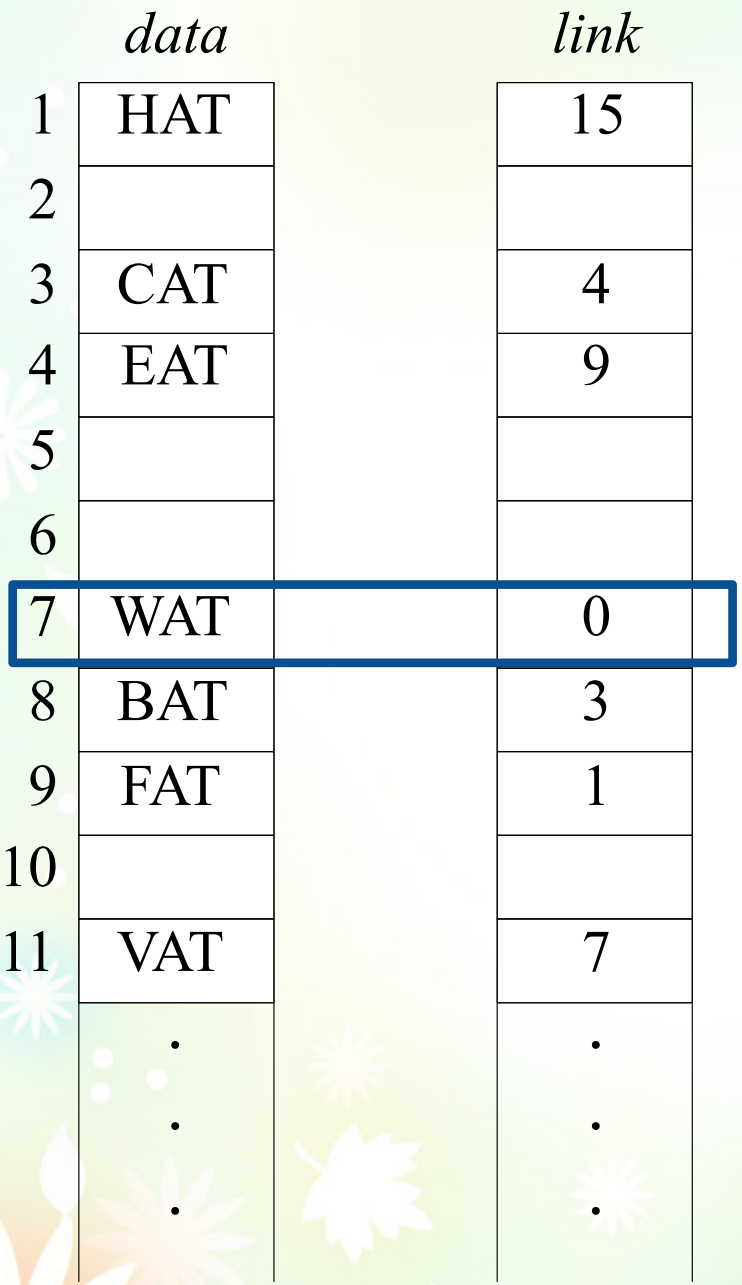
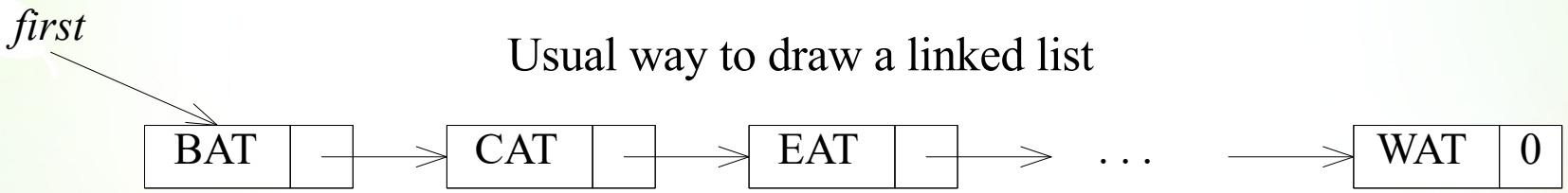


Fig. 4.1 Nonseq. list

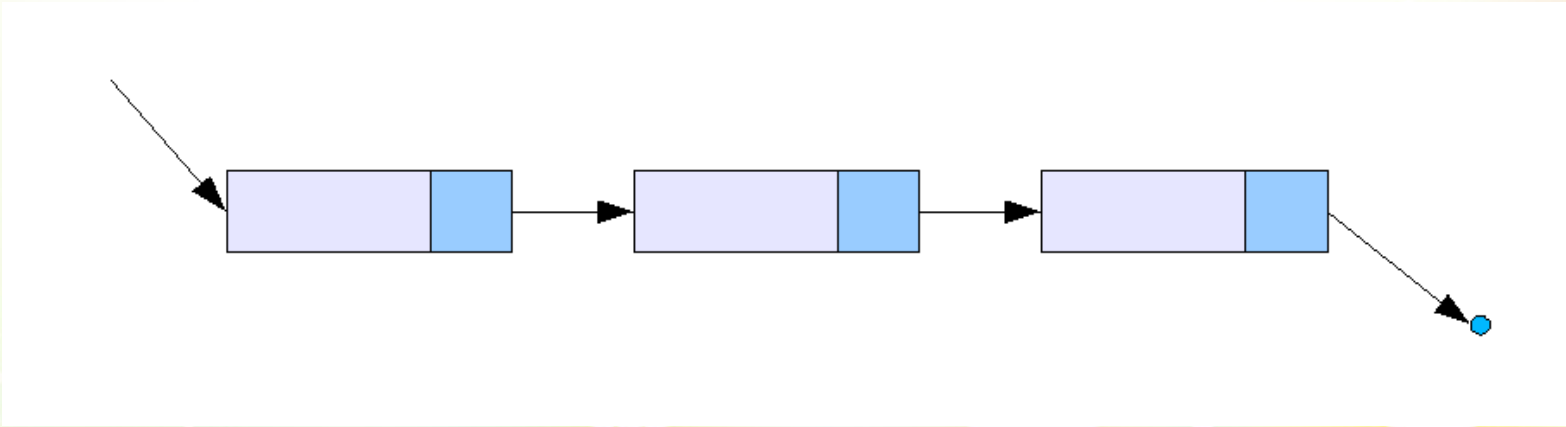
- The elements of the list are stored in a one-dimensional array called ***data***, but the elements no longer occur in a sequential order.
- To remind us of the real order, the second array, ***link***, is added.
- data[i] + link[i]→ node
- The list starts at data[8]=BAT, a variable *first*=8. link[8]=3 means the location of next data element at 3. Therefore the next data element is data[3]=CAT….
- Finally…

- It is customary to draw linked list as an ordered sequence of nodes with links being represented by arrows.



- (1) the nodes do not actually reside in sequential locations
- (2) the actual locations of nodes are immaterial
- Therefore, when we write a program that works with lists, we do not look for a specific address except we test for zero

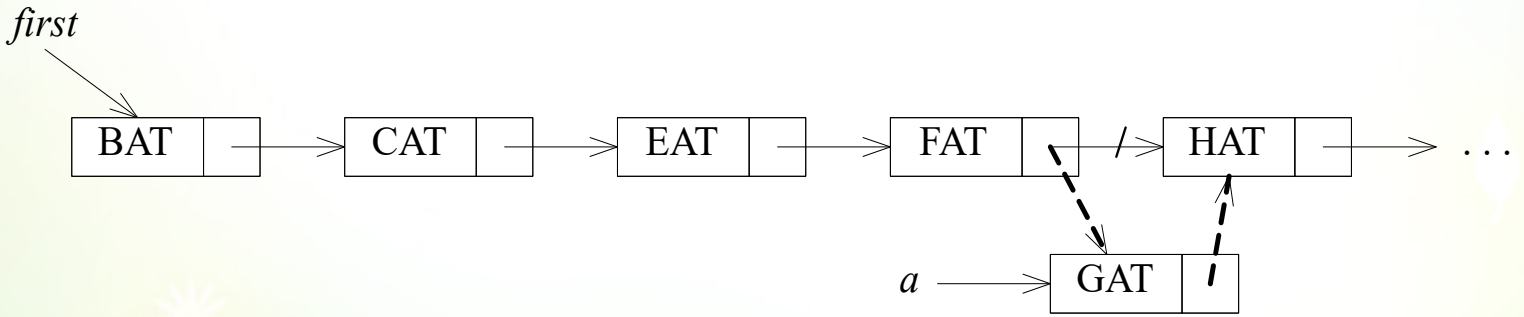
- Fig. 4.1 and 4.2 are called singly linked lists (單一鏈結串列) or chains(鏈)
- In a singly linked list, each node has **exactly one pointer field**. A chain is a singly linked list that is comprised of **zero or more nodes**.



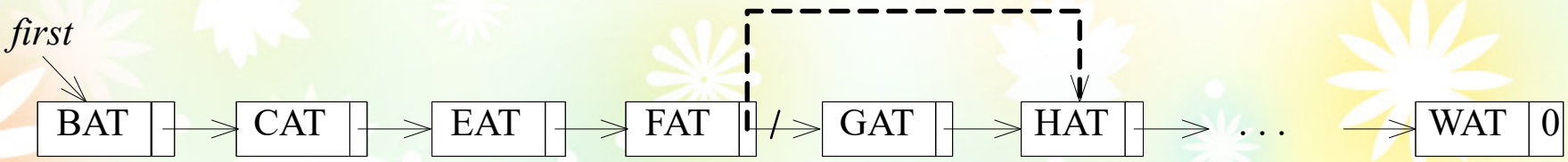
- Let us see why it is easier to make insertions and deletions at arbitrary position using a linked list rather than a sequential list
- To insert GAT between FAT and HAT:
  - (1) Get a node a that is currently unused.
  - (2) Set the data field of a to GAT
  - (3) Set the link field of a to point to the node after FAT, which contains HAT
  - (4) Set the link field of the node containing FAT to a
- To delete GAT from the list:
  - Find the element that precedes GAT, which is FAT
  - Set link[9] to the position of HAT is 1
- Though the link of GAT still contain a pointer to HAT, it's fine.

	data	*link*
1	HAT	15
2		
3	CAT	4
4	EAT	9
5	GAT	1
6		
7	WAT	0
8	BAT	3
9	FAT	1
10		
11	VAT	7

(a) 把 GAT 插入到 *data* [5]中



(b) 把 GAT 節點插入到串列中



Linked Lists

- The drawbacks of using sequential storage to represent stacks and queues :
  - fixed amount storage.
  - overflow.

- Fixed amount storage :
  - A fixed amount of storage remains allocated to the stack or queue even when the structure is actually using a smaller amount.
- Overflow :
  - Fixed amount of storage may be allocated, thus introducing the possibility of overflow.

儲存空間不是太大就是太小

使用陣列實作線性串列(問題)

除了這二項問題：
- **fixed amount storage.**
- **overflow.**

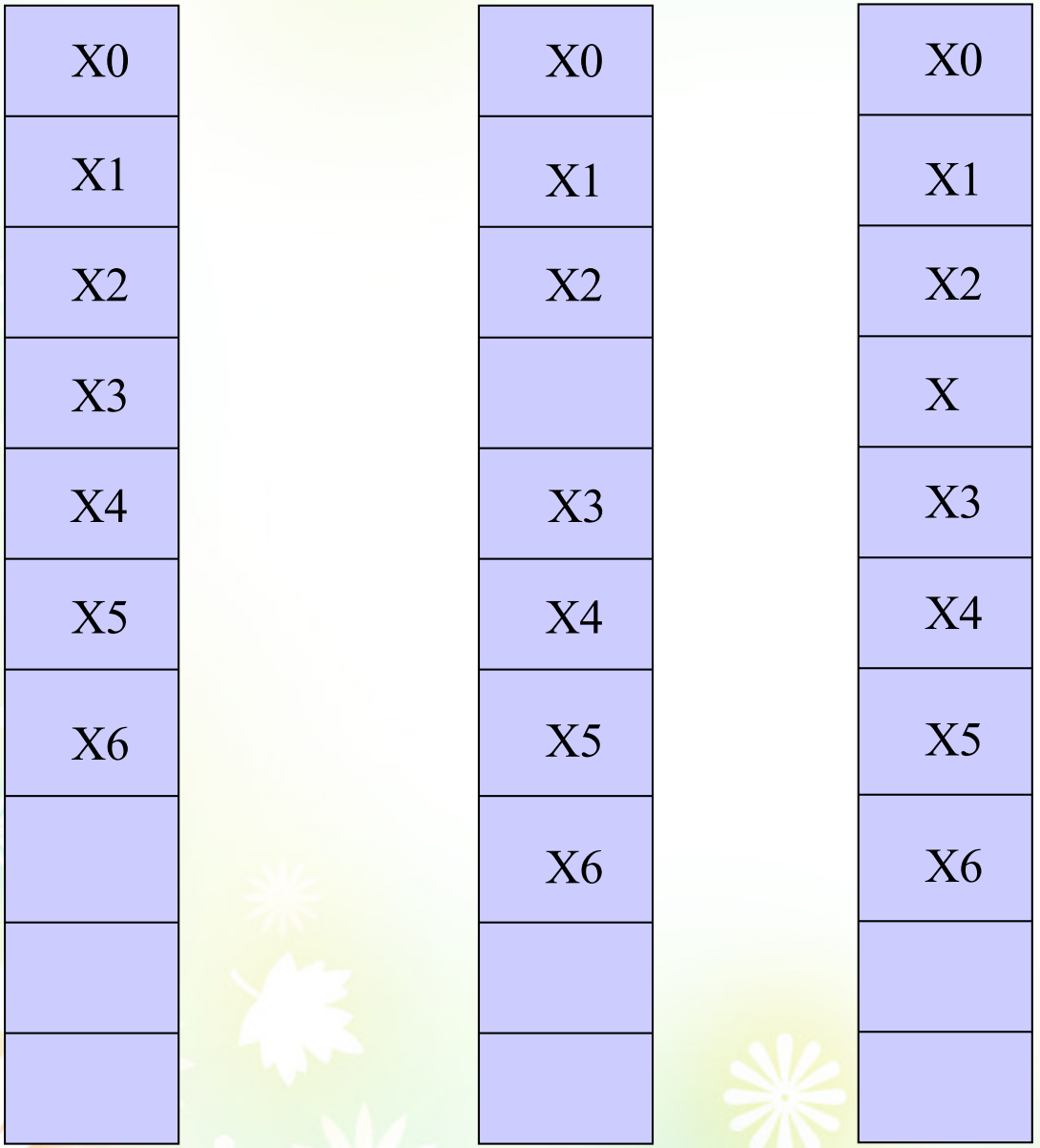
- **複雜的新增與刪除運算：**

在新增或刪除名單時，因為陣列儲存的是循序且連續資料，所以在陣列中需要搬移大量元素，才能滿足同縣市位在同一區段。例如：在桃園市新增江小魚，則王小美、李光明和周星星都需要依序往後搬移1個元素，才能將江小魚插入，同理，刪除王大毛時，之後的所有元素也需要往前搬移。

- **浪費記憶體空間：**

因為並不知道名單有多少位，所以需要宣告一個很大的結構陣列來儲存名單，如果最後只使用到幾個元素，就會造成大量記憶體空間的閒置。

Comparison of an array and a list:

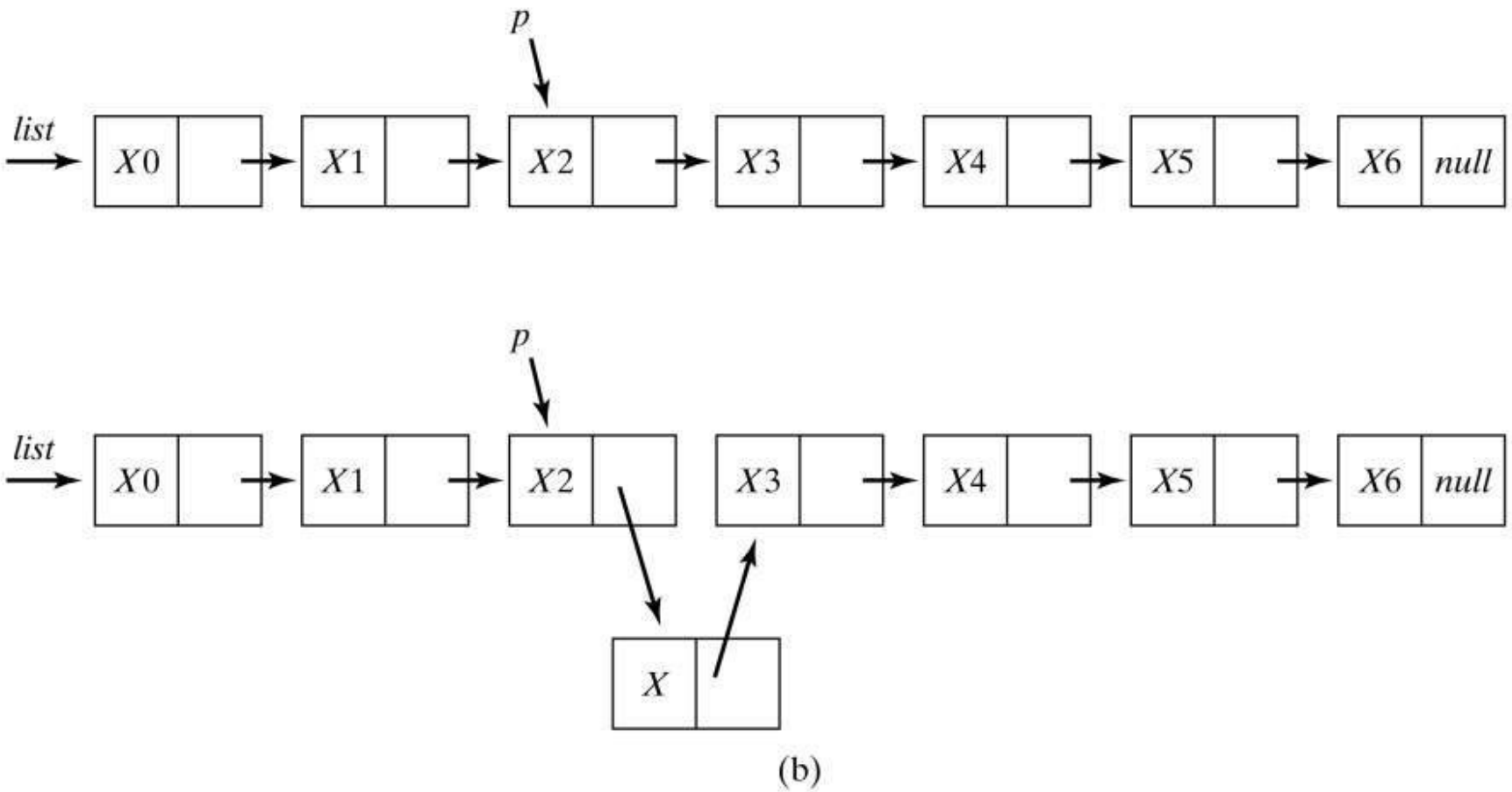


(a)

將 X 插入在 X2 及 X3 之間要移動 4 個元素
如果此陣列有 500 個元素，則必須更動一大堆的元素

相當浪費時間

而且萬一原來陣列就已經全滿，就會造成 overflow的情形



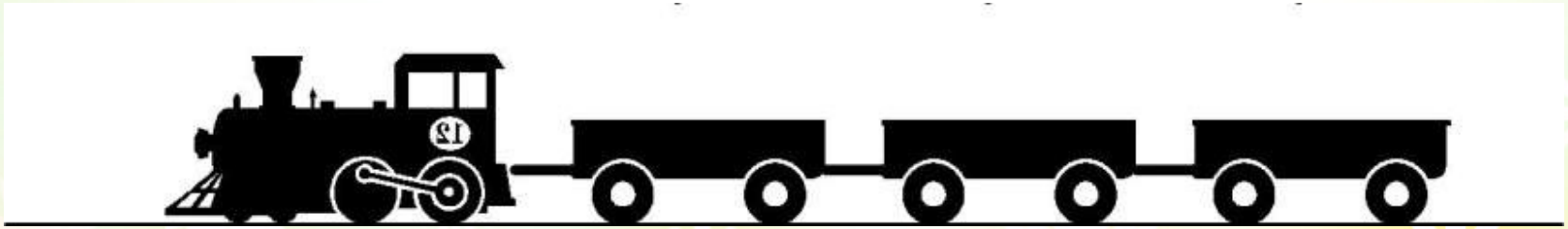
(b)

Figure 4-2-5(b)

Inserting an element into a list.

使用鏈結串列實作線性串列

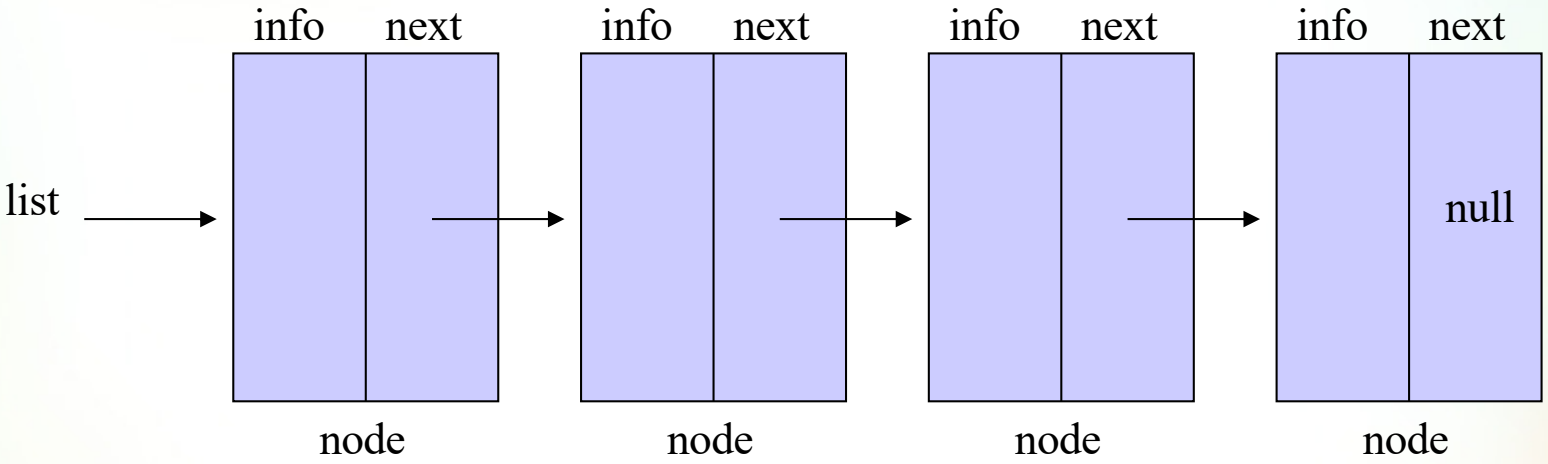
- 「鏈結串列」（Linked Lists）是一種實作線性串列的資料結構。在現實生活中，鏈結串列如同火車掛車廂以線性方式將車廂連結起來，每個車廂是鏈結串列的節點（Nodes），儲存線性串列的資料，車廂和車廂之間的鏈結，就是節點間的鏈結（Link）。



THE LINKED LIST

Each node contains the address of its successor (in addition to whatever data it holds).

Ex:



A ***null pointer*** is used to signal the end of a list.

Assume

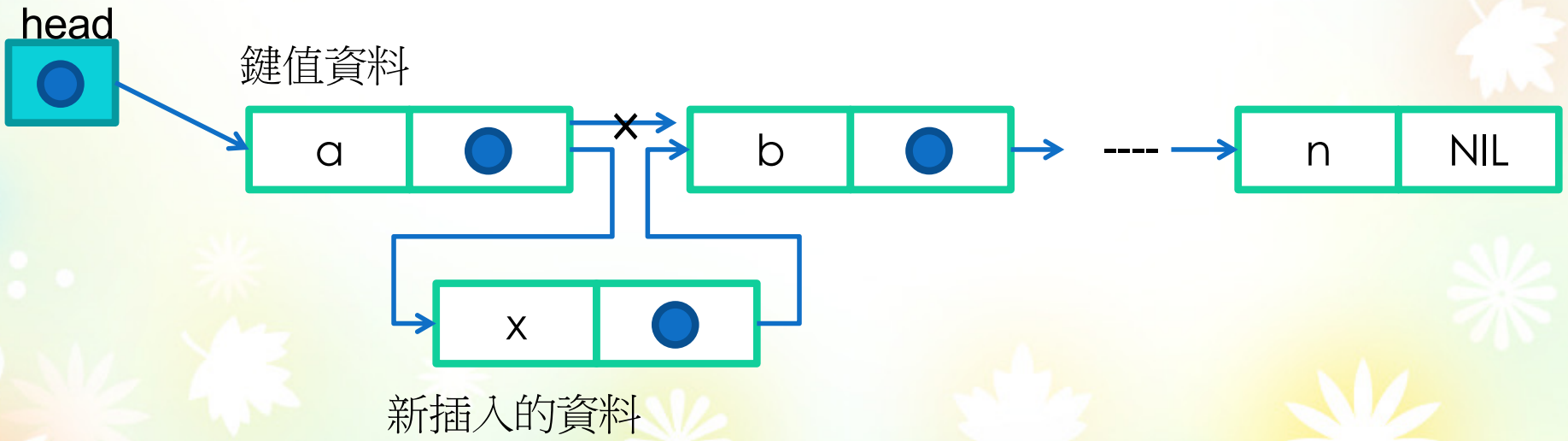
p = getnode ( )

returns the address of an available node.

A node may be inserted onto the front of a list:

Implementation- using **struct & C++ function**

- 建立串列後，將資料插入串列
  - 若是陣列之類的資料結構插入或刪除現有資料，必須要移動一部份的陣列元素
  - 串列則只需要替換指標即可



```c
#include "stdafx.h"
#include <stdio.h>
#include <malloc.h>
#include <string.h>
#include <stdlib.h>
struct tfield {
    char name[20];          /* 姓名*/
    char tel[20];           /* 電話號碼*/
    struct tfield *pointer;
 /* 指向下筆資料的指標*/
} *head;
struct tfield *talloc(void);
void genlist(void); //串列產生
void displist(void); //顯示串列
void link(char *);
int _tmain(int argc, _TCHAR* argv[])
{char key[32]; // for input from monitor
    genlist();     displist();
    while (printf("Key Name "),
          scanf("%s",key)!=EOF)
   { link(key);}
    displist();
    system("pause");   return 0;
}
```

```c
void genlist(void)       /* 產生串列*/
{
    struct tfield *p;
    head=NULL;
    while (p=talloc(), scanf("%s %s",p->name,p->tel)!=EOF)
    {
        p->pointer=head;
        head=p;
    }
}
```

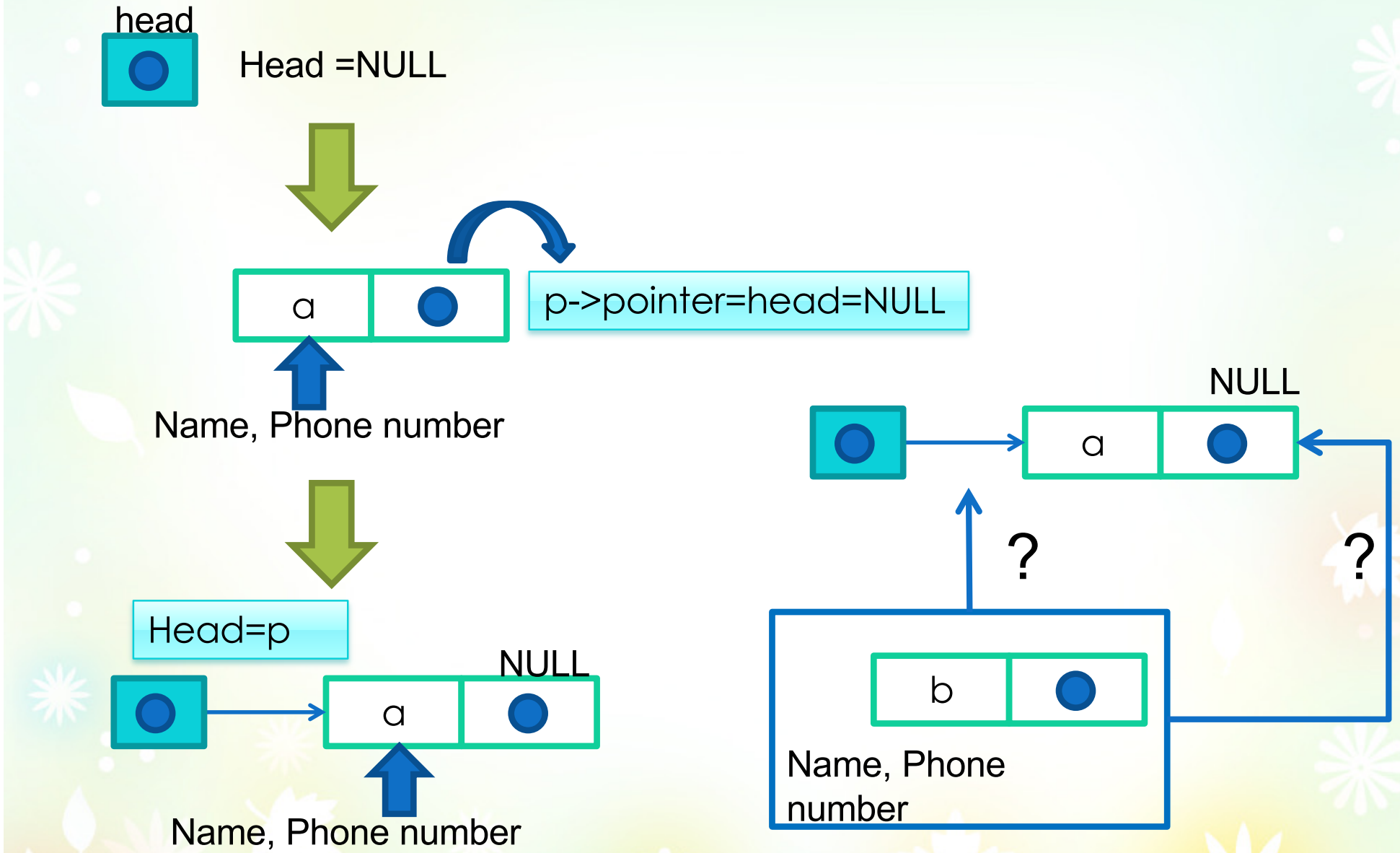
動態宣告產生一個結構

這段很重要！！

```c
void displist(void)      /* 顯示串列*/
{
    struct tfield *p;
    p=head;
    while (p!=NULL){
        printf("%15s%15s\n",p->name,p->tel);
        p=p->pointer;
    }
}
```

使用動態記憶體函式malloc取得struct tfield的記憶體區間

```c
struct tfield *talloc(void)     /* 取得記憶體區域*/
{
    return (struct tfield *)malloc(sizeof(struct tfield));
}
```


head
Head =NULL
a
p->pointer=head=NULL
Name, Phone number
Head=p
NULL
a
Name, Phone number
NULL
a
?
?
b
Name, Phone number
想想看，若按照程式寫法
新資料應該是從哪裡放進去？

新增串列，必須先告訴程式，新資料n要插入哪一個資料(key)的**後面**

```
void link(char *key)    /* 新增串列*/
{
   struct tfield *p,*n;
   n=talloc();
   printf("新增資料");
   scanf("%s %s",n->name,n->tel);
   p=head;
   while (p!=NULL){
      if (strcmp(key,p->name)==0){
         n->pointer=p->pointer;
         p->pointer=n;
         return;
      }
      p=p->pointer;
   }
   printf("找不到鍵值\n");
}
```

Control + Z is used to signal an end-of-file